Firewall Rule Analyzer

Audits a firewall rulebase for security problems. You export your rules to CSV, point the analyzer at them, and get back a single self-contained interactive HTML report: scored findings, a zone traffic matrix, risk charts, and every rule expandable for detail.

Features

- **Multi-vendor** — Palo Alto, FortiGate, Cisco ASA, Check Point, Juniper SRX, plus a generic fallback. Always pass `--vendor` explicitly; see the auto-detection warning below.
- **Security analysis** — insecure protocols (Telnet, FTP, SMBv1, SNMP, NetBIOS, POP3/IMAP, MySQL and more), overly permissive rules, remote-management exposure, rules that *allow* known-malicious IP feeds, and missing logging.
- **Rule optimization** — duplicate rules, shadowed rules, subnet overlaps, and hidden-allow ordering problems.
- **Hygiene checks** — unused rules, stale rules, expired rules, deny-any-any cleanup rule presence and position, and security-profile coverage.
- **Application awareness** — resolves service and application names against the Palo Alto Applipedia database for ports and 0–5 risk ratings.
- **Zone-based analysis** — maps rule addresses onto configured zones and builds a full zone-to-zone traffic matrix.
- **Transparent scoring** — every finding carries its score, a breakdown, and the formula used to reach it. The report includes a scoring methodology section.
- **Web interface** — run and review analyses from a browser instead of the command line. See Web interface.

Installation

Prerequisites

PROF

- Python 3.8 or higher
- No runtime dependencies — the analyzer uses only the standard library. (Chart.js and DataTables are pulled from a CDN by the generated report.)

Setup

```
git clone https://github.com/ashmxwy/firewall_analyzer.git
cd firewall_analyzer

# Create and activate a virtual environment
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

# Install in editable mode
pip install -e .
```

That puts a `firewall-analyzer` command on your PATH. To run without installing:

```
PYTHONPATH=src python3 -m firewall_analyzer.cli --help
```

If you also want the standalone helper in `tools/`, install the extra:

```
pip install -e ".[tools]"        # adds pandas + openpyxl
```

Web interface

Everything below can be done from a browser instead of the command line — upload a rule export, watch it run, and work through the findings. It is the same analysis engine, so the numbers match the CLI exactly.

```
pip install -e ".[ui]"
firewall-analyzer-ui
```

That starts the interface on http://127.0.0.1:8000 and opens a browser tab. There is no container runtime, database server or build step involved — it is Python plus static files.

`/` is the landing page; `/app` is the application, behind a sign-in.

Installing on another machine

Everything the platform needs is in the repository, including the 6.3 MB Palo Alto Applipedia database, so there is nothing to download separately. The clone transfers about 3 MB and lands as roughly 14 MB on disk.

PROF

```
git clone -b gui https://github.com/ashmxwy/firewall_analyzer.git
cd firewall_analyzer
./setup.sh
```

That is the whole thing. `setup.sh` checks Python, installs the package into your user account, and opens the web interface in a browser — printing the four account passwords the first time.

On Windows the script does not apply — see Installing on Windows.

There is **no virtual environment to activate**. Everything lands in `~/.local`, nothing is written under `/usr`, and `sudo` is never needed. After the first run `firewall-analyzer-ui` works from any directory.

> **The `-b gui` matters.** The web interface lives on the `gui` branch; `main` is older and has no `webui/` and no `[ui]` extra, so a plain `git clone` installs with `WARNING: does not provide the extra 'ui'` and leaves you with no `firewall-analyzer-ui` command. If you have already cloned without it, `setup.sh` notices and offers to switch branches for you.

Command	Does
`./setup.sh`	Install if needed, then open http://127.0.0.1:8000
`./setup.sh --install-only`	Set everything up without starting
`./setup.sh --port 9000`	Start on another port
`./setup.sh --share`	Listen on all interfaces so the team can reach it
`./setup.sh --reset-passwords`	New passwords for the four accounts, then exit
`./setup.sh --uninstall`	Remove the package again; leaves `var/` alone

Re-running is safe: accounts are only created on the very first run.

The default passwords are `Admin!2345`, `Exec!2345`, `Manager!2345` and `Analyst!2345`. Choose your own by exporting them first — they apply only on the first run, when the accounts are created:

```
FWA_PASSWORD_ADMIN='...' FWA_PASSWORD_ANALYST='...' ./setup.sh
```

A note on `--break-system-packages`

PROF

Debian, Ubuntu 23.04+ and Fedora mark the system Python "externally managed" (PEP 668), so pip refuses to install outside a virtual environment. Since this installs with `--user`, everything goes to `~/.local/lib` and `~/.local/bin` and nothing under `/usr` is touched — the flag only acknowledges the marker. The script detects this and adds it automatically, and prints what it is doing.

Undo it at any time with `./setup.sh --uninstall`.

If you would rather keep it isolated after all, a virtual environment still works and needs no flag:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[ui]"
firewall-analyzer-ui
```

Installing on Windows

`setup.sh` is a shell script and does not run natively on Windows. Open **PowerShell** in the repository folder and run these instead:

```
git checkout gui
git pull

pip install --user -e ".[ui]"

python -m firewall_analyzer.webui.cli --reset-passwords

python -m firewall_analyzer.webui.cli
```

The third command prints the four account passwords — copy them, they are shown once. The fourth starts the server and opens a browser at http://127.0.0.1:8000.

Four things differ from Linux, and each one bites if you get it wrong:

No `--break-system-packages`	That flag answers PEP 668, which is a Linux packaging rule. Windows Python is not externally managed, so plain `--user` works. Add the flag only if pip actually asks for it
Keep the quotes on `".[ui]"`	PowerShell treats bare `[...]` as syntax, so `pip install -e .[ui]` fails there. Quoted, it is literal
`python`, not `python3`	Windows installs the interpreter as `python`. If that is not found, use the launcher: `py -m firewall_analyzer.webui.cli`
Use the module form	The `firewall-analyzer-ui` command lands in `%APPDATA%\Python\Python3xx\Scripts`, which is usually not on `PATH`. `python -m firewall_analyzer.webui.cli` is the same program and does not depend on `PATH`

To choose the passwords yourself rather than take generated ones, set them first — PowerShell uses `$env:` rather than a prefix:

```
$env:FWA_PASSWORD_ADMIN='Admin!2345'
$env:FWA_PASSWORD_EXECUTIVE='Exec!2345'
$env:FWA_PASSWORD_MANAGER='Manager!2345'
$env:FWA_PASSWORD_ANALYST='Analyst!2345'
python -m firewall_analyzer.webui.cli --reset-passwords
```

PROF

Everything else behaves identically: the data directory, the four roles, the activity log and the report composer are all the same.

Installing by hand

```
git clone -b gui https://github.com/ashmxwy/firewall_analyzer.git
cd firewall_analyzer
pip install --user --break-system-packages -e ".[ui]"   # drop the flag
if pip does not ask for it
firewall-analyzer-ui
```

If `firewall-analyzer-ui` is not found afterwards, `~/.local/bin` is missing from your `PATH`. Either add it, or run the same thing as a module, which does not depend on `PATH` at all:

```
python3 -m firewall_analyzer.webui.cli
```

Accounts and roles

There is no sign-up. On first run the launcher creates one account per role and prints the generated passwords **once** — only the hashes are stored, so capture them then. Set them yourself with `FWA_PASSWORD_ADMIN`, `FWA_PASSWORD_EXECUTIVE`, `FWA_PASSWORD_MANAGER`, `FWA_PASSWORD_ANALYST`.

Role	Dashboard preset	Run analyses	Configuration & History
Administrator	any of the three, switchable	yes	yes
Security Manager	Security Manager	yes	yes
Analyst	Analyst	yes	yes
Executive	Executive (locked)	no	no

Additional roles can be created at runtime — see Creating a new role.

Only the administrator sees the Executive / Security Manager / Analyst switcher on the dashboard — every other role is pinned to its own preset, so the control would be a no-op. An Executive gets the posture and the findings, not the machinery: Configuration and History are hidden, and the server returns 403 on any write, so hiding the controls is presentation rather than the protection itself.

Sessions are an HMAC-signed HttpOnly cookie, valid for 12 hours (`FWA_SESSION_TTL`). The signing key is generated once and kept at

`session.key` in the data directory, so restarts don't sign everyone out.

Lost the passwords?

Seeding runs **once** — it deliberately skips if any account already exists, so a restart never resets everyone's password. That also means the printed passwords are the only copy. To recover:

```
firewall-analyzer-ui --reset-passwords            # generate and print
new ones

FWA_PASSWORD_ADMIN='...' \
  firewall-analyzer-ui --reset-passwords          # or choose them
yourself
```

It creates any missing account, re-hashes the rest, prints once and exits without starting the server. Existing sessions stay valid until they expire — delete `session.key` as well to force everyone out immediately.

Creating a new role

Roles are rows in the database, not constants in the code, so an administrator can add one without a deployment. **Users & Roles** → **Roles** → **New role**, or via the API:

```
curl -X POST http://127.0.0.1:8000/api/admin/roles \
  -H 'Content-Type: application/json' -b cookies.txt \
  -d '{
    "name": "auditor",
    "label": "External Auditor",
    "sections": ["dashboard", "analysis"],
    "view": "manager",
    "may_write": false,
    "may_switch_view": false,
    "description": "Read-only access for an external audit team."
  }'
```

A role definition is four grants:

Field	Meaning
`sections`	Which sidebar groups appear. Any of `dashboard`, `analysis`, `configuration`, `history`
`view`	Which dashboard preset the holder lands on: `executive`, `manager` or `analyst`

Field	Meaning
`may_write`	Whether holders can upload and run analyses, and delete them
`may_switch_view`	Whether holders may change the dashboard preset. Normally only Administrator

It takes effect on the holder's next page load — no restart. The four built-in roles can be edited but not deleted, and the rules below are enforced server-side:

- the last administrator cannot be demoted or deleted;
- a role still held by an account cannot be deleted;
- an unknown section name is rejected rather than silently granting nothing.

To grant administration itself, add `administration` to the role's sections.

Who did what

Users & Roles → **Activity log** records sign-ins (including *failed* ones, with the username attempted), sign-outs, analyses run and deleted, and every account or role change — each with the actor, their role, the source IP, the outcome and a detail line. It is filterable by event type, by actor, and to denied events only. Entries survive deletion of the account that made them.

Flag	Purpose
`--port 9000`	Use a different port (the next free port is picked automatically if it is taken)
`--host 0.0.0.0`	Share with your team over the network instead of local-only
`--no-browser`	Do not open a browser window
`--reload`	Reload on code changes, for development

What it does

- **Upload CSV or Excel.** `.xlsx` is converted server-side, so the format that comes out of the firewall works without a conversion step.
- **Overview** — three dashboards, one per audience, all fed by a single `/api/analyses/{id}/dashboard` call so they never disagree:
  - *Executive* — the overall risk rating, the key risk areas in plain language, compliance for every framework assessed, and recommended next steps.
  - *Security Manager* — remediation workload by category, **remediation impact** (posture points recovered by fixing each area), cleanup candidates, and control status per framework.
  - *Analyst* — a severity × category heatmap, the rule-score distribution, `any` and clear-text-protocol exposure, and zone-to-zone flows.

Score-1 findings are pass records (a profile is present, the cleanup rule is in place), so "what needs fixing" figures exclude them; severity totals keep them, to match the Findings page. Every panel links through to the filtered findings or rule behind it.

- **Compliance** — one tab per framework chosen for the run. Several can be chosen in the first step of Run New Analysis; see Compliance frameworks.
- **Findings** — every finding, filtered by severity and category, searchable, with the score breakdown behind each one.
- **Rule Explorer** — every rule in the file, including disabled rules the analyzers skip, so "why isn't this flagged?" has an answer on screen.
- **Zone Analysis**, **External Communications**, **Logging & Audit** — the focused views that previously meant scrolling the HTML report.
- **Full Report** — compose a client-ready report: tick the sections you want, see the result rendered in the page, then download it as a self-contained HTML file or print it to PDF. The engine's own generated report is still reachable underneath, unchanged, for continuity with the CLI.
- Findings export as CSV or JSON from any screen.

Overall risk rating

The headline rating uses the same four levels as a finding — **Critical, High, Medium, Low** — and is computed in one place (`core/posture.py`) for the CLI, the HTML report and the web UI alike.

1. **Posture score, 0–100.** Each assessed rule counts once, by its most severe open issue, weighted relative to critical (critical 1.0, high 0.5, medium 0.2, low 0.05). `score = 100 × (1 − weighted share)`.
2. **Rating.** 85+ Low · 70–84 Medium · 50–69 High · below 50 Critical. A floor then applies: any critical issue rates the rule base at least High, any high issue at least Medium — so one exposed rule cannot vanish inside a large clean policy, and one finding does not decide the whole rating either.

Weights, bands and floors live under `posture_rating` in `config/analysis_config.json`. Each run stores the parameters it used, so a dashboard reproduces that run's rating even after the config changes. Analyses from before this scale are re-rated from their own findings on startup; the old label is kept as `legacy_health`.

Compliance frameworks

Frameworks are a registry (`analysis/frameworks.py`) that the engine, the API (`/api/meta` → `frameworks`), the upload wizard and the Compliance page all

read. One run can be assessed against several; each is stored and shown separately, and one framework failing never fails the run.

Key	Framework	Status
`ecc`	NCA Essential Cybersecurity Controls (ECC-1:2018)	Implemented
`ccc`	NCA Cloud Cybersecurity Controls	Planned — selectable, recorded, not assessed
`nist`	NIST Cybersecurity Framework	Planned — selectable, recorded, not assessed

To implement a planned framework, subclass `ComplianceFramework`, implement `assess(rules)` returning the result contract documented at the top of `analysis/frameworks.py` (ECC in `analysis/compliance.py` is the reference), and swap its `PlannedFramework` entry in `REGISTRY` for the new class. No frontend change is needed: the wizard card, the Compliance tab, the dashboard panels and the report section all pick it up.

Via the API, repeat the `frameworks` form field (or comma-separate it): `-F frameworks=ecc -F frameworks=nist`. Unknown keys are rejected with 400.

Where state lives

Source checkout	`var/` beside the repo (gitignored)
Installed copy	`~/.firewall-analyzer/`
Override	`FWA_DATA_DIR=/some/path`

Uploads, generated reports and a SQLite database live there. The Applipedia database is found automatically at `data/palo_alto_applipedia.json`; set `FWA_APPID_DB` to point elsewhere. Nothing leaves the machine.

PROF

The UI always loads the packaged scoring config, so application-risk findings are included — it is not affected by the `analysis_config.json` shadowing described below.

Usage

The analyzer takes two positional arguments: **an input CSV and an output HTML path**.

```
firewall-analyzer --vendor fortinet rules.csv reports/report.html
```

Input must be CSV

Excel files are **not** supported — a `.xlsx` produces a confusing UTF-8 decode error. Convert first:

```
python3 -c "import pandas as pd;
pd.read_excel('rules.xlsx').to_csv('rules.csv', index=False)"
```

For example, to run against the FortiGate test fixture in this repo:

```
python3 -c "import pandas as pd;
pd.read_excel('fortigate_policy_analyzer_test_240.xlsx').to_csv('rules.c
sv', index=False)"
mkdir -p reports
firewall-analyzer --vendor fortinet rules.csv reports/report.html
```

`reports/` is gitignored, so it's a good place to put output.

Common invocations

```
# Also export the findings as JSON
firewall-analyzer --vendor fortinet --export-json reports/analysis.json
\
                  rules.csv reports/report.html

# Consolidate findings by rule name in the report
firewall-analyzer --vendor fortinet --group-findings rules.csv
reports/report.html

# Write out the default config templates to edit
firewall-analyzer --vendor fortinet --export-configs ./myconfigs
rules.csv reports/report.html

# Show parser decisions and full tracebacks
firewall-analyzer --vendor fortinet --debug rules.csv
reports/report.html
```

PROF

Always pass `--vendor`

Omitting `--vendor` falls back to header-based auto-detection, which is **currently unreliable**. `SchemaNormalizer.detect_vendor` tests the Cisco ASA indicators with `any()` rather than `all()`, so a single `Protocol` column is enough to claim the file — and it is tested *before* FortiGate. A normal FortiGate export therefore parses as Cisco ASA, with the wrong field mappings throughout:

```
--vendor fortinet   ->  216 rules analyzed, 12 disabled excluded, 933
findings
auto-detected       ->  228 rules analyzed,  0 disabled excluded, 821
findings
```

The disabled-rule column is never read, so disabled rules are audited as if they were live.
Pass `--vendor` until `detect_vendor` is fixed.

Command line options

```
usage: firewall-analyzer [-h]
              [--vendor
{palo_alto,cisco_asa,fortinet,checkpoint,juniper_srx,generic}]
              [--field-mapping FIELD_MAPPING] [--zone-config
ZONE_CONFIG]
              [--analysis-config ANALYSIS_CONFIG] [--appid-db APPID_DB]
              [--export-json EXPORT_JSON] [--export-configs
EXPORT_CONFIGS]
              [--group-findings] [--high-risk-threshold
HIGH_RISK_THRESHOLD]
              [--medium-risk-threshold MEDIUM_RISK_THRESHOLD] [--debug]
              input_csv output_report

positional arguments:
  input_csv             Path to input CSV file
  output_report         Path to output HTML report file

options:
  -h, --help            show this help message and exit
  --vendor              Firewall vendor type (default: auto-detect from
headers)
  --field-mapping       Path to JSON field mapping configuration
  --zone-config         Path to JSON zone configuration
  --analysis-config     Path to JSON analysis scoring configuration
  --appid-db            Path to Palo Alto AppID database JSON
                        (default: data/palo_alto_applipedia.json, then
./palo_alto_applipedia.json)
  --export-json         Also export analysis results to this JSON file
  --export-configs      Write default configuration templates to this
directory and exit
  --group-findings      Group findings by rule name in the report
  --high-risk-threshold Rule score at or above which a rule is high risk
(default: 70)
  --medium-risk-threshold
                        Rule score at or above which a rule is medium
risk (default: 40)
  --debug               Enable debug output and full tracebacks
```

The process exits `0` on success and `1` on error. Findings do **not** affect the exit code.

Input format

Column names are matched per vendor via the field mapping config, case- and separator-insensitive. Palo Alto example:

```
Name,Source Zone,Destination Zone,Source Address,Destination
Address,Application,Service,Action,Disabled,Log Setting
Allow_Web,untrust,web,any,web_servers,web-browsing,http,allow,no,log-all
```

FortiGate exports are matched on `PolicyID`, `RuleName`, `SrcZone`, `DstZone`, `SourceAddress`, `DestinationAddress`, `Service`, `Action`, and optionally `Enabled`, `HitCount`, `LogTraffic`, `ExpiryDate`, and the `*Profile` columns.

Multi-value cells may be separated by `;`, `,`, `|`, or newlines. Actions are normalized across vendors, so `accept`/`permit`/`pass` all become `allow` and `drop`/`reject`/`block` all become `deny`.

Output

The HTML report is self-contained apart from two CDN scripts. `--export-json` additionally writes a machine-readable summary:

```
{
  "timestamp": "2026-09-07T11:05:00",
  "vendor_type": "fortinet",
  "summary": {
    "overall_health": "Critical",
    "critical_issues": 116,
    "high_risk_issues": 172,
    "total_issues": 829
  },
  "findings": [
    {
      "rule_id": "1",
      "rule_name": "USERS_TO_WAN_01_SSH",
      "issue_type": "overly_permissive",
      "risk_level": "low",
      "score": 35,
      "description": "Rule is overly permissive with 'any' in:
destination addresses",
      "recommendation": "Specify exact destination servers or networks
rather than broad ranges. Consider implementing Zero Trust principles."
    }
  ],
  "findings_by_rule": {}
}
```

Top-level keys are `timestamp`, `vendor_type`, `summary`, `statistics`, `zone_summary`, `findings`, and `findings_by_rule`.

Configuration

Three JSON files drive the analyzer. Generate editable copies with `--export-configs`:

File	Flag	Purpose
`field_mapping_schema.json`	`--field-mapping`	Maps each vendor's CSV columns onto the internal rule model
`zone_config.json`	`--zone-config`	Maps subnets to named zones for the traffic matrix
`analysis_config.json`	`--analysis-config`	Risk weights, thresholds, and application risk scoring

Known gotcha: `analysis_config.json` shadowing

`ConfigurationManager` looks for `analysis_config.json` in the **current working directory** before falling back to the copy bundled in the package. The `analysis_config.json` checked in at the repo root is a minimal template with `application_risk_scoring.enabled` set to `false`, so running from the repo root **silently disables application risk scoring**:

```
firewall-analyzer --vendor fortinet rules.csv reports/report.html
# -> 829 findings, 0 of them application risk

firewall-analyzer --vendor fortinet \
    --analysis-config src/firewall_analyzer/config/analysis_config.json
\
    rules.csv reports/report.html
# -> 933 findings, 104 of them application risk
```

PROF

Pass `--analysis-config` explicitly, or run from a directory without a root `analysis_config.json`, until the two config copies are consolidated.

Project layout

```
firewall_analyzer/
├── src/firewall_analyzer/
│   ├── cli.py                  # argparse entry point
│   ├── orchestrator.py         # runs the analyzers, aggregates scores
and stats
│   ├── core/models.py          # FirewallRule, SecurityFinding, enums
│   ├── config/manager.py       # loads the three JSON configs
│   ├── data/                   # CSV parsing, schema normalization,
```

```
Applipedia lookup
│   ├── analysis/               # the individual security checks
│   ├── reporting/
│   │   ├── html_generator.py   # builds the report
│   │   └── assets/             # report.css and report.js
│   ├── utils/ip_utils.py       # subnet and IP helpers
│   └── webui/                  # web interface (FastAPI + static files)
│       ├── cli.py              # firewall-analyzer-ui entry point
│       ├── main.py             # app, routes, static mounts
│       ├── runner.py           # drives the analyzer in-process
│       ├── store.py            # SQLite persistence
│       ├── auth.py             # roles, password hashing, session
cookie
│       ├── jobs.py             # background analysis pool
│       └── static/             # plain HTML/CSS/JS, no build step
├── data/palo_alto_applipedia.json
├── tools/check_browsing_profiles.py
├── docs/                       # Sphinx sources
└── reports/                    # generated output (gitignored)
```

Development

Building documentation

```
cd docs
make html
```

Web UI

The frontend is plain ES modules and CSS under `src/firewall_analyzer/webui/static/` — no bundler, no npm, no build step. Edit a file and reload. `firewall-analyzer-ui --reload` also restarts the API on Python changes.

Charts are hand-rolled inline SVG in `static/js/charts.js` rather than a chart library, which keeps the app self-contained and working offline. The severity palette is a reserved status palette — those colours never double as series colours, and every severity is shown with its label so meaning never rests on hue alone.

Editing the report's CSS and JavaScript

The report's stylesheet and script live in `src/firewall_analyzer/reporting/assets/` as plain `.css` and `.js` files. Edit them directly.

Render-time data reaches the JavaScript through a single `REPORT_CONFIG` global emitted by `_get_enhanced_javascript()`; the colour palette reaches the CSS as custom properties on

`:root` emitted by `_get_css_styles()` from `EnhancedHTMLReportGenerator.colors`. Add a new value to one of those two methods rather than interpolating Python into the asset files.

```
cd ~/Desktop/PwC/firewall_analyzer
PYTHONPATH=src python3 -c "
import getpass
from firewall_analyzer.webui import auth, store
pw = getpass.getpass('New admin password: ')
assert len(pw) >= 8, 'use at least 8 characters'
user = store.get_user_by_username('admin')
store.set_password(user['id'], auth.hash_password(pw))
print('admin password changed')"
```